



Waipapa
Taumata Rau
**University
of Auckland**

Classification

SOFTENG 753: Machine Learning Techniques
and Applications

➤ Semester 2, 2026

➤ Maryam Hemmati
m.hemmati@auckland.ac.nz

Content



- **Classification Terminology**
- **Binary Classification**
- **Classifying Movie Reviews**

➤ Classification Terminology

➤ **Sample** or **Input**

- One data point that goes into our model.

➤ **Prediction** or **Output**

- What comes out of our model.

➤ **Target**

- The truth. What our model should ideally have predicted, according to an external source of data.

➤ **Prediction Error** or **Loss Value**

- A measure of the distance between our model's prediction and the target.

➤ Classification Terminology

➤ **Classes**

- A set of possible labels to choose from in a classification problem.
- When classifying cat and dog pictures, "dog" and "cat" are the two classes.

➤ **Label**

- A specific instance of a class annotation in a classification problem.
- If picture #1234 is annotated as containing the class "dog," then "dog" is a label of picture #1234.

➤ **Ground-Truth** or **Annotations**

- All targets for a dataset, typically collected by humans.

➤ Classification Terminology

➤ Binary Classification

- A classification task where each input sample should be categorized into **two exclusive** categories.

➤ Categorical Classification or Multiclass Classification

- A classification task where each input sample should be categorized into **more than two** categories
- Classifying handwritten digits.

➤ Multilabel Classification

- A classification task where each input sample can be assigned **multiple labels**.
- A given image may contain both a cat and a dog and should be annotated with both the "cat" label and the "dog" label.
- The number of labels per image is usually **variable**.

➤ Classification Terminology

➤ Mini-Batch or just Batch

- A small set of samples (typically between 8 and 128) that are **processed simultaneously** by the model.
- The number of samples is often a **power of 2**, to facilitate memory allocation on GPU.
- When training, a mini-batch is used to compute a **single gradient-descent update** applied to the weights of the model.

➤ Binary Classification

- **Two-class** classification, or **binary** classification, is one of the most common kinds of machine learning problem.
- We will learn to classify movie reviews as **positive** or **negative**, based on the text content of the reviews.
- We will use **IMDb dataset**
 - A set of **50,000 highly polarized reviews** from the **Internet Movie Database**.
 - They are split into **25,000** reviews for **training** and **25,000** reviews for **testing**.
 - Each set consisting of **50% negative** and **50% positive** reviews.

➤ Classifying Movie Reviews

- The **IMDb dataset** comes **packaged** with **Keras**.
 - Like the MNIST dataset.
- It has already been **preprocessed**.
 - The reviews (sequences of words) have been turned into **sequences of integers**.
 - Each **integer** stands for a **specific word in a dictionary**.
 - This enables us to focus on model building, training, and evaluation.
- When we run the following code the first time, about **80 MB** of data will be downloaded.

```
from keras.datasets import imdb

(train_data, train_labels), (test_data, test_labels) = imdb.load_data(
    num_words=10000
)
```

Loading **IMDb** dataset.

➤ Classifying Movie Reviews

- The argument `num_words=10000` means we will only keep the **top 10,000 most frequently occurring** words in the training data.
- Rare words will be **discarded**.
 - This allows us to work with vector data of **manageable size**.
 - Without this limit, we would be working with **88,585 unique** words in the training data, which is **unnecessarily large**.
 - Many of these words **only occur in a single sample** and cannot be meaningfully used for classification.
- The variables `train_data` and `test_data` are NumPy arrays of reviews.
 - Each review is a list of **word indices** (encoding a sequence of words).
- `train_labels` and `test_labels` are NumPy arrays of 0s and 1s.
 - Where **0** stands for **negative** and **1** stands for **positive**.

➤ Classifying Movie Reviews

- We can decode one of these reviews back to English words.

```
# word_index is a dictionary mapping words to an integer index.
word_index = imdb.get_word_index()
# Reverses it, mapping integer indices to words
reverse_word_index = dict([(value, key) for (key, value) in word_index.items()])
# Decodes the review. Note that the indices are offset by 3 because 0,
# 1, and 2 are reserved indices for "padding," "start of sequence," and
# "unknown."
decoded_review = " ".join(
    [reverse_word_index.get(i - 3, "?") for i in train_data[0]]
)
```

Decoding reviews **back to text**.

```
>>> decoded_review[:100]
? this film was just brilliant casting location scenery story direction everyone
```

The **leading ?** corresponds to a **start token** that has been prefixed to each review.

➤ Classifying Movie Reviews – Data Preparation

- The list of integers in different reviews have **different lengths**.
 - We cannot directly feed those lists into a neural network.
 - We have to turn our lists into **tensors**.
- There are **two ways** to prepare our data:
 - **Pad** our lists so that they all have the same length, then turn them into an **integer tensor** of shape **(samples, max_length)**.
 - Start our model with a layer capable of handling such integer tensors (the **Embedding layer**).
 - **Multi-hot encode** our lists to turn them into vectors of 0s and 1s reflecting the presence or absence of all possible words.
 - Turning the sequence [8, 5] into a **10,000-dimensional vector** that would be all 0s except for indices 5 and 8, which would be 1s.

➤ Classifying Movie Reviews – Data Preparation

- We take the second approach to **vectorize the data**.
- We should also vectorize the labels, which is straightforward.
 - Our labels are already NumPy arrays, so just convert the type from **ints** to **floats**.

```
import numpy as np

def multi_hot_encode(sequences, num_classes):
    # Creates an all-zero matrix of shape (len(sequences), num_classes)
    results = np.zeros((len(sequences), num_classes))
    for i, sequence in enumerate(sequences):
        # Sets specific indices of results[i] to 1s
        results[i][sequence] = 1.0
    return results

# Vectorized training data
x_train = multi_hot_encode(train_data, num_classes=10000)
# Vectorized test data
x_test = multi_hot_encode(test_data, num_classes=10000)
```

Encoding the integer sequences via **multi-hot encoding**.

```
y_train = train_labels.astype("float32")
y_test = test_labels.astype("float32")
```

➤ Classifying Movie Reviews – Building Model

- The input data is vectors.
- The labels are scalars (1s and 0s).
- A type of model that performs well on this problem is a plain **stack** of densely connected (**Dense**) layers with **relu** activations.
- Two key **architecture decisions** about such a stack of Dense layers:
 - How many **layers** to use?
 - How many **units** to choose for each layer?

```
import keras
from keras import layers

model = keras.Sequential([
    layers.Dense(16, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(1, activation="sigmoid"),
])
```

Model definition.

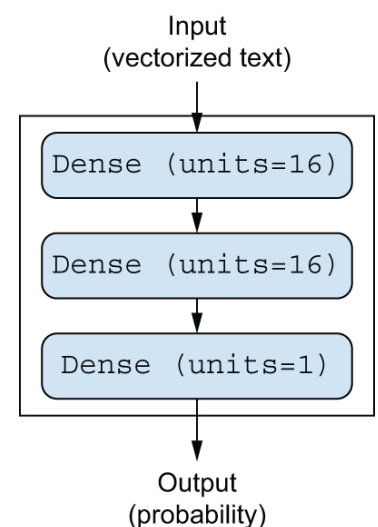
The **first argument** being passed to each Dense layer is the number of **units** in the layer: the **dimensionality of representation space** of the layer.

➤ Classifying Movie Reviews – Building Model

- Our model has
 - Two **intermediate** layers with **16 units** each.
 - A **third layer** that will output the **scalar prediction** regarding the sentiment of the current review.
- Each **Dense** layer with a **relu** activation implements the following chain of tensor operations:

$$\text{output} = \text{relu}(\text{dot}(\text{input}, W) + b)$$

- Weight **matrix W** has shape (**input_dimension**, **16**).
- The dot product with W will **project the input data** onto a **16-dimensional** representation space.
- Then we will add the **bias vector b** and apply the **relu** operation.



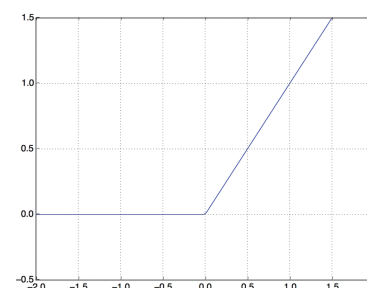
The **three-layer** model.

➤ Classifying Movie Reviews – Building Model

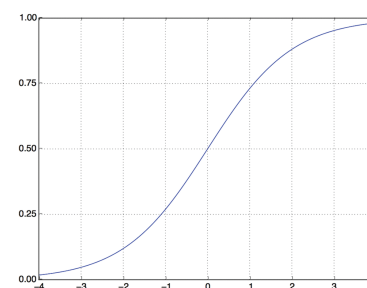
- The **dimensionality** of our **representation space** can be intuitively understood as
 - "How much freedom we are allowing the model to have when learning internal representations."
- Having more units (a higher-dimensional representation space) allows our model to learn more complex representations.
- It also makes the model more computationally expensive and may lead to learning unwanted patterns.
 - Patterns that will improve performance on the training data but not on the test data.

➤ Classifying Movie Reviews – Building Model

- The intermediate layers use **relu** as their **activation function**.
- The final layer uses a **sigmoid activation** to output a probability.
 - A score between 0 and 1, indicating how likely the review is to be positive.
- A **relu** (rectified linear unit) is a function meant to zero-out negative values.
- A **sigmoid** "squashes" arbitrary values into the **[0, 1] interval**
 - Outputs something that can be interpreted as a **probability**.



The **rectified linear unit** function.



The **sigmoid** function.

➤ Classifying Movie Reviews – Activation

- Without an **activation function** (also called a **non-linearity**), the Dense layer would consist of two **linear operations** of a dot product and an addition.
 - The layer could only learn **linear transformations** (affine transformations) of the input data.
- The **hypothesis space** of the layer would be the set of all possible **linear transformations** of the input data into a 16-dimensional space.
 - Such a hypothesis space is too **restricted**.
 - It would **not benefit** from **multiple layers** of representations because a deep stack of linear layers would still implement a linear operation.
- For a much **richer hypothesis space** that would benefit from **deep representations**, we need a **non-linearity** or **activation function**.
 - **relu** is the most popular activation function in deep learning.
 - There are many other candidates: **prelu** (Parametric Rectified Linear Unit), **elu** (Exponential Linear Unit), and so on.

➤ Classifying Movie Reviews – Compiling Model

- We need to choose a **loss function** and an **optimizer**.
- We have a **binary classification** problem and the output of our model is a **probability**.
 - We end our model with a **single-unit layer** with a **sigmoid activation**.
 - It is best to use the **binary_crossentropy** loss.
- Other viable choices like **mean_squared_error** are also viable.
 - But **crossentropy** is usually the best choice when we are dealing with models that output probabilities.
- **Crossentropy** is a quantity from the field of information theory that **measures the distance between probability distributions**.
 - In this case, between the ground-truth distribution and our predictions.

➤ Classifying Movie Reviews – Compiling Model

- We will choose **adam** (Adaptive Moment Estimation) optimizer.
- Adam optimizer **adjusts learning rates** during training, and is usually a good default choice for virtually any problem.

```
model.compile(  
    optimizer="adam",  
    loss="binary_crossentropy",  
    metrics=["accuracy"],  
)
```

Compiling the model.

- We will monitor **accuracy** during training.

➤ Classifying Movie Reviews – Validating Data

- We use a “**validation set**” to **monitor** the accuracy of the model during training.
- The results we get on the validation set inform our next choices to **improve** training.
 - What **model size** to use.
 - How **many epochs** to train for.
- We perform the **final evaluation** round in a completely unbiased way using the **test set**.
 - Test set is a set of **never-before-seen** samples.

```
x_val = x_train[:10000]  
partial_x_train = x_train[10000:]  
y_val = y_train[:10000]  
partial_y_train = y_train[10000:]
```

Setting aside a **validation** set.

➤ Classifying Movie Reviews – Training Model

- We train the model
 - **20 epochs**
 - 20 **iterations** over **all samples** in the training data.
 - **Mini-batches** of **512 samples**.
- At the same time, we monitor **loss** and **accuracy** on the **10,000** samples we set apart.
 - By passing the validation data as the **validation_data** argument to **model.fit()**.
- At the end of every epoch, there is a slight pause as the model computes its **loss** and **accuracy** on the **10,000** samples of the validation data.

```
history = model.fit(
    partial_x_train,
    partial_y_train,
    epochs=20,
    batch_size=512,
    validation_data=(x_val, y_val),
)
```

Training the model.

➤ Classifying Movie Reviews – Training Model

- Instead of **manually splitting** out validation data from your training data and passing it as the **validation_data** argument, we can also use the **validation_split** argument in **fit()**.
 - It specifies a **fraction** of the training data to use as validation data.

```
history = model.fit(
    x_train,
    y_train,
    epochs=20,
    batch_size=512,
    validation_split=0.2,
)
```

- In this example, **20%** of the samples in the **x_train** and **y_train** arrays are being held out from training and used as **validation data**.

➤ Classifying Movie Reviews – Evaluating Model

- The call to `model.fit()` returns a `History` object,
 - This object has a member `history`, which is a `dictionary containing data` about everything that happened during training.

```
>>> history_dict = history.history
>>> history_dict.keys()
dict_keys(['accuracy', 'loss', 'val_accuracy', 'val_loss'])
```

- The dictionary contains **four entries**:
 - One per **metric** that was being monitored during **training** and during **validation**.
 - **Training Loss**
 - **Validation Loss**
 - **Training Accuracy**
 - **Validation Accuracy**

➤ Classifying Movie Reviews – Evaluating Model

- We use `Matplotlib` to plot
 - The **training** and **validation loss**
 - The **training** and **validation accuracy**

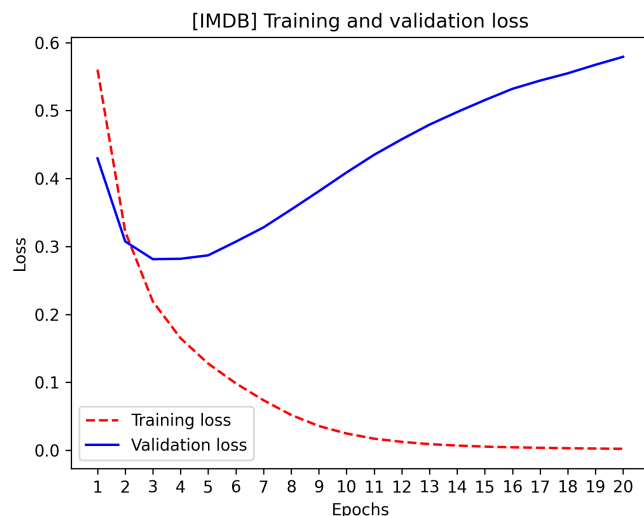
```
import matplotlib.pyplot as plt

history_dict = history.history
loss_values = history_dict["loss"]
val_loss_values = history_dict["val_loss"]
epochs = range(1, len(loss_values) + 1)
# "r--" is for "dashed red line."
plt.plot(epochs, loss_values, "r--", label="Training loss")
# "b" is for "solid blue line."
plt.plot(epochs, val_loss_values, "b", label="Validation loss")
plt.title("[IMDB] Training and validation loss")
plt.xlabel("Epochs")
plt.xticks(epochs)
plt.ylabel("Loss")
plt.legend()
plt.show()
```

Plotting the **training** and **validation loss**.

➤ Classifying Movie Reviews – Evaluating Model

- The training loss **decreases** with every epoch.
 - That is what we expect when running **gradient-descent optimization**.
 - The quantity we are trying to **minimize** should be less with every **iteration**.
- The **validation** loss seems to **peak** at the **fourth epoch**.
 - A model that performs better on the training data **is not necessarily** a model that will do better on data it has never seen before.
- **Overfitting**
 - After the fourth epoch, we are **overoptimizing** on the training data.
 - We end up learning representations that are **specific** to the training data and don't **generalize** to data outside of the training set.



The **training** and **validation** loss.

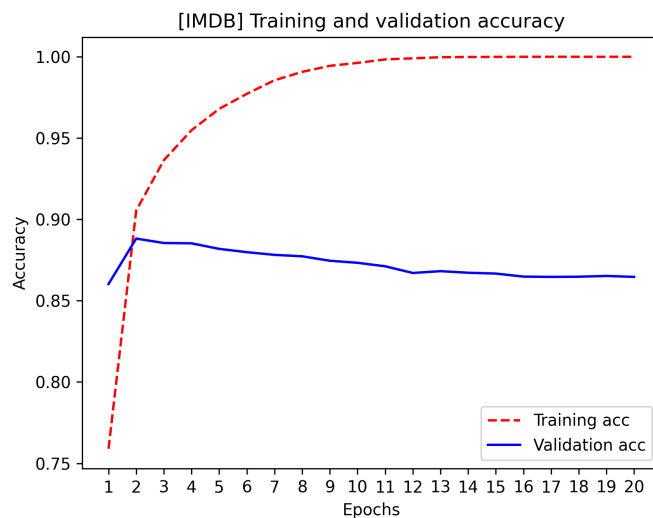
➤ Classifying Movie Reviews – Evaluating Model

```
# Clears the figure
plt.clf()
acc = history_dict["accuracy"]
val_acc = history_dict["val_accuracy"]
plt.plot(epochs, acc, "r--", label="Training acc")
plt.plot(epochs, val_acc, "b", label="Validation acc")
plt.title("[IMDB] Training and validation accuracy")
plt.xlabel("Epochs")
plt.xticks(epochs)
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

Plotting the **training** and **validation** accuracy.

➤ Classifying Movie Reviews – Evaluating Model

- The training accuracy **increases** with every epoch.
- The validation accuracy **decreases** after a point due to **overfitting** problem.
 - We can **stop training** after four epochs to prevent overfitting
 - We can use a range of techniques to **mitigate overfitting**.



The **training** and **validation** accuracy.

➤ Classifying Movie Reviews – Training Model

- We train a new model from scratch for **four epochs** and then evaluate it on the test data.
- This fairly naive approach achieves an accuracy of **88%**.
- With **state-of-the-art approaches**, we should be able to get close to **95%**.

```
model = keras.Sequential(  
    [  
        layers.Dense(16, activation="relu"),  
        layers.Dense(16, activation="relu"),  
        layers.Dense(1, activation="sigmoid"),  
    ]  
)  
model.compile(  
    optimizer="adam",  
    loss="binary_crossentropy",  
    metrics=["accuracy"],  
)  
model.fit(x_train, y_train, epochs=4, batch_size=512)  
results = model.evaluate(x_test, y_test)
```

Training the model for **four** epochs.

➤ Classifying Movie Reviews – Predictions

- We want to use our trained model in a [practical setting](#).
- We can generate the [likelihood of reviews being positive](#) by using the [predict](#) method.

```
>>> model.predict(x_test)
array([[ 0.98006207]
       [ 0.99758697]
       [ 0.99975556]
       ...,
       [ 0.82167041]
       [ 0.02885115]
       [ 0.65371346]], dtype=float32)
```

- The model is [confident](#) for some samples
 - 0.99 or more -> **Positive**
 - 0.01 or less -> **Negative**
- It is [less confident](#) for others
 - 0.6 or 0.4

Classifying Movie Reviews

Let's have a look and try the example in [Jupyter notebook](#)

➤ Classification – Homework

- Try using **one or three representation layers** and see how doing so affects validation and test accuracy.
 - We used two representation layers before the final classification layer.
- Try using layers with **more units** or **fewer units**: 32 units, 64 units, and so on.
- Try using the **mean_squared_error** loss function instead of **binary_crossentropy**.
- Try using the **tanh** activation instead of **relu**.
 - The **tanh** was popular in the early days of neural networks.

Summary

- We learned about the specialized terms used in classification.
- We looked at Binary Classification and started with an example to classify movie reviews using IMDb dataset.
- We covered the full pipeline of preparing data, compiling, training, evaluating the model, and finally predicting with our trained model.



References

- The content and figures used in these slides are extracted from the textbook **Deep Learning with Python, Third Edition - François Chollet.**

Next Session: Classification and Regression



Any Questions?